

Scheduling a Wireless Network in Natural Language: Scope Guardrails and Error Attribution in a Validator-Backed Agent

Anonymous Author(s)

Abstract

Running an aggregation scheduler by hand means writing a configuration, invoking a solver and checking its output; we replace that with a sentence. We present a language agent for one-shot aggregation scheduling in multi-hop wireless networks, in which the model interprets the request while scope, parameter legality, topology binding, scheduling and validation are carried out by deterministic components, one a validator implemented separately from the scheduler. Over three repeats behind a frontier model, the agent returns the correct action on all but one or two of the 44 executable requests, matches 94.6% of the contract fields a request fixes, emits no schedule the validator rejected, and acts when it should not on 2 of 144 requests.

To separate the agent’s failures from its scaffolding’s we seal a 144-request benchmark written by two language models and cross-adjudicated blind, half of whose requests ask for capabilities the system lacks. In the configurations we evaluate, the scope gate added for safety lowers false acceptance for a 14B agent at the cost of action accuracy, but behind the frontier agent it costs action accuracy and field accuracy and *raises* false acceptance, buying only fewer model calls, because the gate is matched on the capability’s vocabulary and the agent is not. Demoting it to an advisory notice the model may overrule recovers five items for the 14B agent at an unchanged false-acceptance count. Of the gated frontier arm’s errors, 11 per run are forced by deterministic stages rather than chosen by the model, and two pipeline failures produced no error at all, one hiding an entire agent capability behind an apparent model limitation.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; *Natural language processing*; • **Software and its engineering** → Software verification and validation.

Keywords

LLM agents; tool use; verifiable computation; agent evaluation; benchmark; safety gating; scope detection; wireless scheduling

ACM Reference Format:

Anonymous Author(s). 2027. Scheduling a Wireless Network in Natural Language: Scope Guardrails and Error Attribution in a Validator-Backed Agent. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 12 pages.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

1 Introduction

One-shot aggregation scheduling in a multi-hop wireless network is a solved computational problem, but running the solver is a small, repetitive and error-prone job. An engineer decides the channel count, the collision model, the interference ratio, the seed and the latency target, writes them into a configuration, runs the scheduler, and checks that the schedule it produced is actually collision-free. The scheduling is solved; the interface is not. A natural-language front end would replace the configuration file with a sentence such as “*Plan one collection on topo100_seed0 with three channels and give me the slot count.*” Building one, however, requires answering a question the solver never faced: given an arbitrary sentence, whether the request is in scope, whether it is specified well enough to act on, and whether its parameters are legal. Deciding any of the three wrongly is worse than having no agent at all. A solver handed a configuration assumes someone already decided it. A system that confidently schedules something the user did not ask for, using a tool that faithfully computes whatever it is handed, is a liability rather than a convenience.

We answer that question architecturally, by separating three jobs a single-agent design runs together. The model *interprets*: it turns the request into a typed scheduling request and proposes an action. Deterministic rules *constrain* that proposal: three of them — scope, parameter legality, topology binding — read the raw text and can terminate the request before the model is consulted, a fourth compares the model’s fields against the user’s own wording, and an *EXECUTE* the rules do not sustain does not execute. The solver and the validator *compute and certify*: any schedule reported was produced by the deterministic scheduler and accepted by a validator sharing no code with it. So the model is not cut out of the semantic decision, and Sections 5.2 and 5.3 measure how much of it the model makes under three settings of the gate’s authority; what it cannot do is compute, certify or report. Figure 1 shows the pipeline, described in Section 3.4.

Evaluating such a system requires a benchmark it cannot have been tuned on and a record of what happened inside it, because the score belongs to the whole pipeline rather than to the model. We seal a benchmark of 144 requests (Section 4) and log every stage of every run. Sealing fixes what the benchmark is and proves it did not move, but it is not a held-out set; Section 4.4 says what our reuse of it does and does not support. Behind a frontier model, over three repeats, the agent returns the correct action on 42 to 43 of the 44 executable requests, matches 94.6% of the contract fields a request fixes, emits no schedule the validator rejected, refuses 70 to 71 of the 72 unsupported requests, and acts when it should not on 2 of 144.

Three results carry the paper, and they are our contributions beyond the agent itself. First, what a pre-model scope gate buys depends on the agent behind it. It costs both agents action accuracy; what reverses is safety. For the 14B agent it lowers false acceptance,

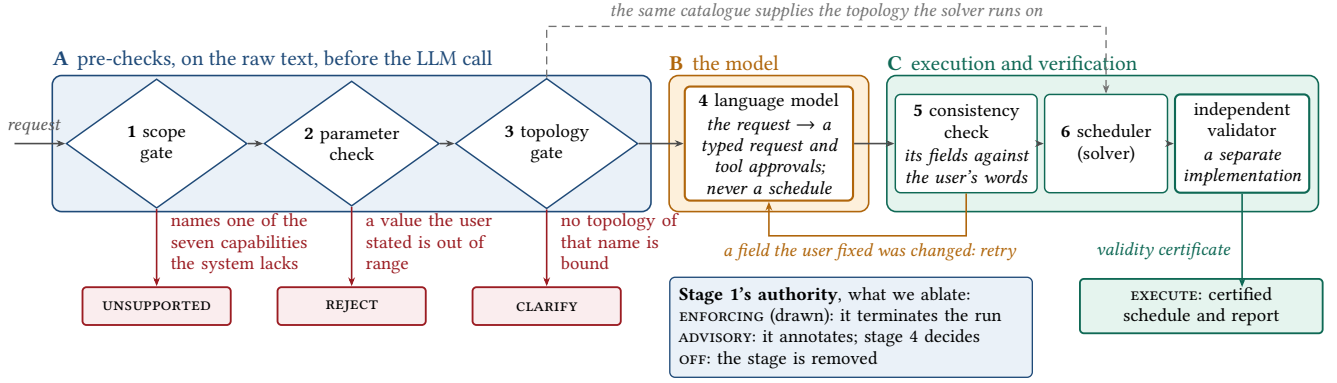


Figure 1: The agent. Lane A’s three checks read the raw request before the language model is called, so the requests they terminate are answered identically whatever model sits behind the agent, each with the action drawn below it and the reason beside the arrow. Lane B is the only model-dependent stage, and returns a typed scheduling request, a proposed action and its tool approvals, never a schedule; a field it changed against the user’s own wording is caught in lane C and the model is asked again. Lane C computes the schedule and certifies it with a validator that shares no code with the scheduler. The dashed line marks the one artefact two lanes share; the inset, the three settings of stage 1’s authority, drawn here ENFORCING (Sections 5.2 and 5.3). Colour repeats the lane labels and means nothing else.

the trade a safety gate is meant to offer; for the frontier agent it raises false acceptance and lowers field accuracy too, buying only fewer model calls and lower latency (Section 5.2). The mechanism is that the gate is matched on the capability’s vocabulary and the agent is not: of the 46 requests asking for an out-of-scope capability without naming it, the gate matches none and the agent refuses a mean 44.7. Second, what the gate costs is its *authority* rather than its detection — running the same detector as a notice the model may overrule recovers five items for the 14B agent without raising false acceptance (Section 5.3). Third, error here is not model error alone: of the requests the gated agent gets wrong, 11 per run are forced by deterministic stages, so an answer-only ledger would have reported every one as model error. The same instrumentation surfaced two evaluation failures that produced no visible error at all (Sections 5.6 and 5.7). One arm ran with no schema constraint, because the provider rejected our schema document and the client retried without one; the run succeeded and returned a complete, plausible set of results, and an entire capability — asking a clarifying question — appeared missing from every model we tested until we found that our own output contract had discarded it. Separately, the local server reloaded the same model onto the CPU between two campaigns, changing every timing by an order of magnitude while leaving the actions intact.

Supporting all three is that sealed benchmark, written by two language models, cross-adjudicated blind and reported with its provenance and its per-batch results. We release the agent, whose deterministic core passed validation on 1260 topology-channel configurations of up to 300 nodes, the benchmark, both pipeline revisions with their manifests, every ledger before and after the repair, and the analysis code:

2 Related Work

A recent survey organises agent evaluation by what is evaluated and by how the evaluation is run [8]. Ours sits under reliability and safety, but our point is about the second axis: the evaluation process itself can fail in ways its metrics do not reveal. Most agent benchmarks also assume every task is achievable with the tools provided, so they measure success and not refusal; the benchmark of Section 4 includes requests the system should refuse. The first axis covers behaviour, capability, reliability and safety. The survey’s second axis covers interaction mode, datasets, metrics and tooling. Knowing that edge is the more consequential behaviour for a deployed system: an agent that quietly does the wrong thing is worse than one that declines.

The closest work asks exactly this. Cheng et al. construct infeasible tasks automatically, by masking the tools that successful executions depended on, and report a false-continue rate as high as 73.9% [3]. Their infeasibility is *mechanical* — a tool is removed — while the UNSUPPORTED category we measure is *semantic*: every tool is present, and the request asks, in the user’s own words, for a capability the system was never designed to provide. And where they compare agent architectures, we hold the architecture fixed and ablate one component (Section 5.2), so the difference between two arms is attributable to the guardrail rather than to the agent. Their construction is also automatic, which scales; ours is authored and adjudicated, which does not, but which lets us control phrasing — and phrasing is what Section 5.2 shows to be decisive.

Who decides when a rule and a model disagree is an older question than language agents. In safe reinforcement learning a *shield* enforces a specification over a policy’s actions, either preemptively, by offering the agent only the safe actions, or post-posed, by correcting the action it chose [1]; scaling that construction to several agents at once is open work [2]. Our enforcing and advisory modes are the same distinction at the level of a whole turn. For language

agents it reappears as rails written outside the model and interpreted at runtime [10], and what a rail costs is measured less often than what it prevents. That cost is the one safety filtering pays, refusing safe requests that resemble unsafe ones [11]: ours matches the vocabulary and not the meaning, and Sections 5.2 and 5.3 price it in the agent’s own accuracy.

A second line concerns what a structured-output constraint costs. Tam et al. find that restricting models to formats such as JSON degrades reasoning, the more so as the constraint grows rigid [13]; Geng et al. show across 10,000 real-world schemas that satisfying them remains hard [5]; and Li et al. document a sharper interaction, several open-weight models ceasing to invoke tools while remaining schema-compliant [7]. The failure Section 5.6 reports points the other way: those risks are that a constraint is applied and costs something, ours was that it was *silently not applied*, and the cost surfaced as a capability the agent appeared to lack. We note in passing that our data do not reproduce the tool-call suppression Li et al. report: enabling strict schema constraints raised, rather than lowered, the number of gold-EXECUTE items our frontier arm completed with all four required tools approved.

A third asks whether the model can do the computation at all; a systematic review covers more than a hundred such studies [4]. Closest to ours is Jiang et al.’s benchmark of models on combinatorial problems posed in natural language: strong models produce feasible, near-optimal solutions on small instances, and both feasibility and quality deteriorate as instances grow, with graph-structured problems and bottleneck objectives failing most often [6]. Aggregation scheduling is both. We add one axis: those benchmarks ask whether the model solves the problem, we ask what its output is worth when it *does*. Section 5.5 reports a certified schedule at three hundred nodes 789 times longer than the solver’s on that instance — feasible, far from optimal, and no feasibility check can tell the difference. The schedule got that long by giving nearly every transmission a slot of its own. The checker bounds correctness, and only the solver bounds quality.

On benchmark integrity, Sainz et al. argue that contamination must be measured per benchmark rather than assumed away [12]. Authoring a benchmark keeps a public test set out of a training corpus but introduces its own risk, an author who knows the pipeline, and we handle it the same way, by measuring (Sections 4.4 and 6).

Agents that translate operator intent into network configuration are an active area. In application, WirelessAgent composes perception, memory, planning and action modules for network slicing [14], and Agentic-V2X is architecturally close to our system: a small language model proposes a scheduling policy, a validator checks and repairs it before enforcement, and a lightweight controller executes it [9]. Separating a generative proposer from a deterministic checker is the principle we adopt in Section 3. The convergence suggests it is becoming standard for this class of system.

Neither, however, is evaluated against adjudicated gold labels that include requests the system should refuse; both are evaluated on tasks their systems are meant to perform. That measures how well an agent does its job, not whether it knows which jobs are its own, and it does not allow an error to be attributed to the model rather than the scaffolding.

3 The Agent

3.1 Scheduling Instance

A network is an undirected graph $G = (V, E)$ with a sink $r \in V$, where an edge joins two nodes within communication range d . A one-shot aggregation schedule assigns every node $v \in V \setminus \{r\}$ a slot $t(v) \in \{0, 1, \dots\}$, a channel $c(v) \in \{1, \dots, K\}$ and a receiver $p(v)$, and is *valid* when (i) every non-sink node transmits exactly once, (ii) $\{v, p(v)\} \in E$, (iii) the sink never transmits, (iv) no two transmissions sharing a slot and a channel collide, and (v) a node transmits only after every node it receives from has, $t(u) < t(v)$ whenever $p(u) = v$ and $v \neq r$, so the aggregate reaching the sink covers V . The sink is excluded from (v) because $t(r)$ is undefined. Slots are numbered from zero, so the collection occupies $T = 1 + \max_{v \in V \setminus \{r\}} t(v)$ slots, and the objective is to *minimize* T . Conditions (i)–(v) are what the independent validator of Section 3.4 checks; T is not certified, only measured. Throughout, the interference range equals the communication range, $d_I = d$: two transmissions conflict exactly when their transmitters are communication neighbours of a common receiver. This modelling assumption is held fixed everywhere — the request contract pins the corresponding parameter to 1.0, as does every scalability run — and we claim nothing about other interference regimes.

3.2 Agent Task

Given a request x in free text, the agent must commit to exactly one action $a(x) \in \{\text{EXECUTE}, \text{CLARIFY}, \text{REJECT}, \text{UNSUPPORTED}\}$: execute it, ask for a missing or ambiguous requirement, reject an in-scope request whose parameters are invalid, or declare it unsupported because it asks for a capability the system does not have.

PROBLEM 1. *Given a request contract, a catalogue of named topologies and a deterministic scheduler, construct an agent that returns the correct action on an arbitrary natural-language request and, on EXECUTE, a typed request whose execution an independent validator certifies — and an evaluation that attributes each error to the model or to the scaffolding around it.*

3.3 Interface

The scheduler is ANEX (Active Neighbor Exploitation), a published one-shot aggregation algorithm [15] we treat as a fixed tool. What replaces its configuration file is a sentence, and Figure 2 carries one through the pipeline. Nothing beyond that sentence is supplied: the topology is named rather than uploaded, unstated parameters take their contract defaults, and the fields the user did fix are carried through unchanged or the run fails.

A request can also be a hard negative: “*The earlier analysis prioritized battery life; this calculation optimizes latency alone. On topo250_seed0, use ANEX with four channels.*” Its first clause names a capability the system does not have while the request asks for one it does, and Section 5.2 shows this is exactly the case the guardrail gets wrong.

The agent’s response is a typed scheduling request — four channels, seed 31, the legacy-neighbour collision model, interference ratio 1.0, and no latency target — together with approval for the four deterministic tools. The orchestrator binds the named topology, runs the scheduler, and hands the result to the validator,

the request

On legacy30_seed7, the standard single-round collection please; the latency figure is the part I need.

determined by the request

objective=minimize_latency, workload=one_shot

contract defaults, the request being silent

channels=2, collision_model=legacy_neighbor,
interference_ratio=1.0, latency_target_slots=None, seed=0

returned by the deterministic stages

29 transmissions over 26 slots on 30 sensors; validator certificate valid, 0 violations

Figure 2: One request end to end, from the released ledger (ft3-r0029 on legacy30_seed7, 1 model call, 2.98 s). The model returns the typed request, a proposed action and its tool approvals; of the seven contract fields, the sentence determines the two in the second box and the other five are the contract defaults, which contract.py lists as DEFAULT_FIELDS. It never sees the sensors and never writes a schedule. The schedule this request produced is drawn in main-full.pdf in the supplement, and its every entry is in figure_data/fig6_trace.json there.

which checks that every non-sink node transmits exactly once, that senders and receivers are adjacent, that the sink never transmits, and that no two transmissions in one slot and channel collide under the interference model. Only then is a report assembled. The user sees the schedule, the slot count and the certificate; the model’s own prose never reaches them.

3.4 Pipeline

The architecture follows one principle: *the model interprets and proposes; it never computes, validates, or reports*. It emits a structured scheduling request, a proposed action and a set of approved tool calls; nothing else it produces reaches the output, and the proposal binds nothing a later deterministic stage does not sustain. The schedule is computed by the deterministic scheduler, checked by a validator that shares no code with it, and the final report is assembled by a deterministic reporter from verified values.

A request passes through six stages (Figure 1), the first three reading the raw text and able to terminate the run before the model is called — which is what makes the model-independent behaviour in Section 5 measurable.

- (1) **Scope gate.** An independent policy module matches the *raw* user text, before the model sees it, against a closed taxonomy of seven capabilities the system does not provide: digital twin or emulation, state synchronisation, energy-aware scheduling, battery evolution, periodic aggregation, topology adaptation and other optimisation objectives, each with its patterns in the released scope_policy.py. A match short-circuits the run to UNSUPPORTED. The gate is case-insensitive regular expressions, and that choice is what Section 5.2 measures.
- (2) **Explicit-parameter validation.** Parameters stated literally in the raw text are checked against their domains, and a value outside its domain terminates the run as REJECT. Because this stage reads the raw text rather than the model’s

interpretation, a model that silently repairs an invalid value cannot turn a REJECT into an EXECUTE.

- (3) **Topology gate.** If no validated topology is bound, the run terminates as CLARIFY. Topology availability is the orchestrator’s responsibility, never the model’s.
- (4) **Requirement interpretation.** The model receives the raw request and the fact that a topology is bound, and returns a structured scheduling request, a proposed action and a set of approved tool calls, parsed against a strict schema; a schema or consistency failure triggers a bounded retry.
- (5) **Consistency check.** The structured request is compared field by field against what the raw text stated explicitly: a field the user fixed but the model changed is an error, not a preference. This stage also rejects a model that asks for a topology when one is already bound.
- (6) **Deterministic execution.** An EXECUTE must approve all four required tools; approving fewer is a REJECT. The scheduler runs, the independent validator checks its schedule against the collision model, and a deterministic reporter assembles the answer from measured values.

3.5 Guarantees

Two properties follow from the architecture rather than from the model. First, conditional output soundness: no schedule reaches the user unless a validator implemented separately from the scheduler accepts it, so no schedule violating the conditions of Section 3.1 is reported as valid, provided the validator is correct on those conditions. That proviso is an assumption, not a result — the validator is a second implementation, not a proof — but we test it. Injecting one defect at a time into every certified schedule over the topology catalogue (a dropped or duplicated transmission, a link that is not an edge, a transmitting sink, a collision, an inverted precedence pair, and seven more) gives 945 mutants, all 945 rejected, each with the code for the condition it attacks: evidence the checks fire, not that the conditions are the right ones (twelve operators, per-condition counts in validator_mutation/mutation.json).

Second, a bound on model influence: stages 1–3 are deterministic and read the raw request, so on any item they terminate, every arm *in the same gate mode* returns the same action whatever model it uses — stated that way because changing which items stage 1 terminates is the point of the ablation of Section 5.2. Stage 6 is deterministic given the structured request, which is model-dependent, so it carries no such invariance. The bound is not a corner case: stages 1–3 decide 31 of the 144 requests and account for 11 of the 22 to 23 errors the gated frontier arm makes.

Neither property makes the agent correct: the gate can refuse a request the system does support, the parameter check sees only values stated literally, and the model remains free to misread anything the first three stages let through. Quantifying those failures is what the benchmark of Section 4 is for.

4 Benchmark and Setup

Evaluating the agent of Section 3 requires items whose correct action is known and requests the system cannot serve. Benchmarks

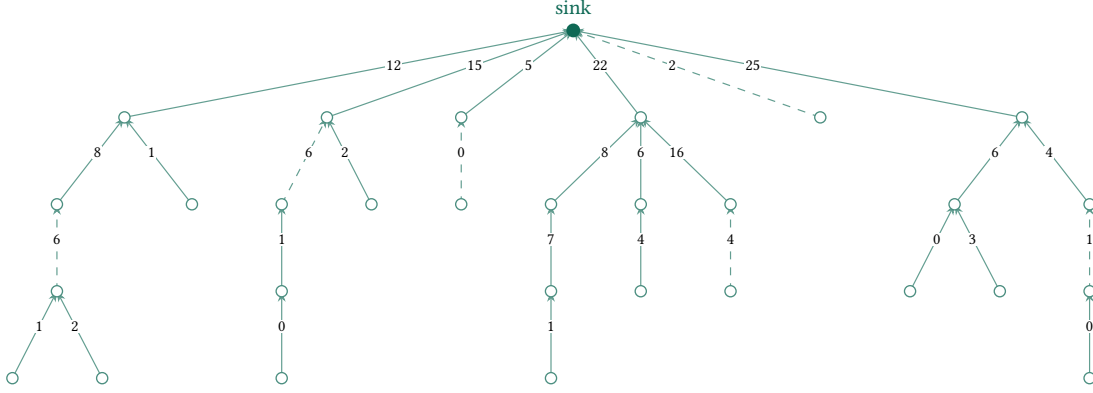


Figure 3: The schedule Figure 2 produced. Each circle is a sensor, placed by its depth in the *aggregation tree* rather than at its position in the network; each arrow is that sensor’s single transmission, the number on it is the slot the transmission occupies, and a solid arrow is channel 1 against a dashed channel 2. Slots are numbered from zero, so they run to 25 for a 26-slot collection.

built around tasks the system is expected to complete cannot measure whether an agent recognises its own boundary, and the infeasibility benchmarks of Section 2 remove a tool rather than ask, in the user’s words, for a capability the tool inventory never had.

4.1 Authoring

Two frontier language models act as authors, independently of each other and, for one of the two batches, of this system’s development; Section 4.4 states the exception and what it bounds. Each receives the same prompt — the request contract, the capability taxonomy, the topology catalogue and the construction rules — and writes 36 *semantic groups*. A group is one intent expressed as two differently worded requests, separating sensitivity to phrasing from sensitivity to meaning. The two batches give 72 groups and 144 requests.

Groups are stratified over ten closed protocol categories, each determining the gold action, with quotas fixed before authoring (Table 1). The ten categories give 44 gold-EXECUTE, 16 gold-REJECT, 12 gold-CLARIFY and 72 gold-UNSUPPORTED requests.

The stratification is deliberately weighted towards UNSUPPORTED: half the benchmark asks for capabilities the system does not have, because that is the case existing benchmarks omit and where an agent’s failure is most consequential.

Three construction rules matter for the results. *Compositional phrasing*: at least a third of the unsupported groups express the out-of-scope capability without its canonical name, so an agent cannot pass by recognising vocabulary. *Hard negatives*: requests mentioning an out-of-scope concept while asking for something supported are gold EXECUTE. *Topology binding by identifier*: a topology is selected only by its exact catalogue identifier appearing as a whole token, so a request describing one without naming it is CLARIFY, one naming an unlisted topology is REJECT, and one naming two binds neither. The catalogue holds seven topologies spanning two communication ranges and 30 to 300 nodes, so range-consistency cases have more than one correct answer.

Table 1: Stratification of the benchmark by protocol category. Quotas are fixed before authoring and checked at seal time; each author writes the “Per author” column, and the two authors’ quotas sum to the “Total” column.

Protocol category	Gold action	Per author	Total
supported_default	EXECUTE	4	8
supported_configured	EXECUTE	4	8
supported_latency_target	EXECUTE	3	6
missing_information	CLARIFY	3	6
invalid_parameter	REJECT	4	8
unsupported_energy	UNSUPPORTED	4	8
unsupported_digital_twin_or_sync	UNSUPPORTED	4	8
unsupported_periodic	UNSUPPORTED	3	6
unsupported_topology_adaptation	UNSUPPORTED	3	6
unsupported_other_objective	UNSUPPORTED	4	8
Groups		36	72
Requests		72	144

4.2 Adjudication

Each batch is blinded — opaque identifiers and request texts, every authoring annotation removed — and annotated independently by the *other* model. Blinding is enforced by the pipeline rather than by convention: a key allowlist rejects any field that could carry a label, and the check runs again before sealing. The two annotations are joined by group and classified automatically under a fixed precedence. The precedence is: a structurally defective item first, then a disagreement about the action, then a paraphrase pair judged non-equivalent, then a disagreement about a request field, and only then full agreement. Of the 72 groups, 63 were in full agreement, 8 were flagged as paraphrase mismatches and 1 as structurally defective; no group drew a disagreement about the gold action. The 9 flagged groups went to a human adjudicator, whose per-group rationale the pipeline records and applies.

4.3 Sealing

Sealing writes a requests-only file for the evaluator and a gold-only file withheld from it, refusing to run if a quota is unmet or a flagged

group is unresolved. Leakage control is mechanical: identifiers are opaque and carry no category, ordering is shuffled with a bound on consecutive same-action runs, and request texts are unique and bound to their gold entry by hash. The evaluator-side file is also checked against a key allowlist for label tokens, source identifiers and schema names. A lock manifest records the SHA-256 of all 76 evaluation-relevant files with the authoring models, decoding settings, scope policy version and contract version.

4.4 Disclosure

Use of language models in this work. Two frontier models are instruments of the experimental design: *ChatGPT-6 Astra Light* (OpenAI, via the ChatGPT desktop application) authored Batch A and blind-adjudicated Batch B, and *Claude Opus 5* (Anthropic, via Claude Code) authored Batch B and blind-adjudicated Batch A, both on 14 September 2026, each receiving only the rendered prompt, which is released verbatim. Separately, language models are the object of study: the arms run phi4:14b, qwen2.5:14b and gpt-5.5-2026-04-23, whose digests, quantization and serving configuration the ledgers record per run. No model originated the hypotheses or the claims; those are the authors’.

One departure from that protocol needs stating, because it bears on how the benchmark is weighed. The session that produced Batch B had also authored the request contract, the prompts, the topology catalogue and the pipeline, and had seen the failure taxonomy from an earlier round. It violates the independence rule of our own protocol and is recorded in the sealed provenance file. Two factors bound it: the blind adjudication was unaffected, Batch B being reviewed by the independent model, which raised all 9 flags against it while Batch A drew none; and most of what that session could exploit is written as an explicit rule in the shared prompt, available to both authors.

We report every result per batch as well as in aggregate, the full table in `analysis_v36/summary.json`. Each half holds 72 requests and no arm separates them by more than four. The direction is consistent within a family rather than across every model arm: all three phi4 configurations and all three frontier ones score higher on Batch B by two to four items, while Qwen, the weakest model arm, leans the other way at -2 , as does the rule baseline at -1 . A contaminated half would predict that sign, and so would Batch B simply being easier for a competent agent; the two are confounded here, and a four-item gap does not separate them, so we do not read the gap’s size as evidence against contamination. Batch A is the half written independently of system development, and reading its column alone removes that exposure — though not the exposure created by repairing the pipeline after seeing failures on the sealed set (Section 5.6). A four-item gap is consistent with Batch B simply being easier, which is what contamination would predict; it is also consistent with a ceiling effect, since an arm near the floor cannot reveal a difficulty difference a strong arm can, and our data cannot separate these. What it is not consistent with is a large contamination effect: the gap is of the same order as the cross-session drift we measure in Section 5.7.

Table 2: Configuration of the eight evaluated arms. All arms see identical requests in identical order and run the same scope detector; the column says what that detector is allowed to do. off marks the two ablation arms, where it is not consulted, and advisory the two arms where it annotates the request and the model decides.

Arm	Model	Scope gate	Serving
RULE	—	enforcing	local
PHI4	phi4:14b	enforcing	Ollama
PHI4-NG	phi4:14b	off	Ollama
PHI4-ADV	phi4:14b	advisory	Ollama
QWEN	qwen2.5:14b	enforcing	Ollama
GPT	gpt-5.5-2026-04-23	enforcing	API
GPT-NG	gpt-5.5-2026-04-23	off	API
GPT-ADV	gpt-5.5-2026-04-23	advisory	API

4.5 Arms

Eight arms run the same 144 sealed requests through the same harness; the first two rows of Table 3 give each one’s model and gate mode. The rule-based arm is a deterministic baseline that never calls a language model; it runs the same stage-1 gate, because a baseline that skipped it would be a different pipeline rather than this one without its model. Three arms differ only in the model behind stage 4: two 14B models served locally and one frontier model over an API. Four more pair each capability level with a changed gate — phi4 and the frontier model, each run once with stage 1 replaced by a stub reporting every request in scope, and once with stage 1 reporting but unable to refuse (Section 5.3). phi4 is the 14B model we ablate because it is the stronger of the two locally served ones here, 64 of 144 against Qwen’s 54, so the ablation is not measuring a gate against a model that fails anyway.

The ablation is implemented by substituting the gate in the orchestrator’s namespace for the duration of one process. No file under the sealed pipeline is modified, the manifest still verifies, and the arm is written to its own ledger under its own name with a distinct policy version, so it cannot be mistaken for the production system in later analysis. Every other component — consistency check, topology gate, scheduler, and independent validator — runs unchanged.

4.6 Decoding and Serving

The local arms decode greedily at temperature 0, with two retries and a 600-second timeout, and each is run three times; because decoding is greedy, those repeats measure whether the serving stack returns the same answer to the same prompt rather than sampling the model. The frontier model accepts only temperature 1, rejecting anything else at the transport layer, so that arm samples where the local arms do not. We flag this asymmetry wherever the arms are compared, because the direction of the bias is not obvious. Sampling can cost accuracy on a task with one correct answer, but it also means the frontier arm is not credited with a determinism the local arms enjoy.

Every arm decodes under a schema constraint, but not from the same document. The local server enforces the agent’s validation schema directly as a generation grammar, while the API’s strict mode accepts only a narrower subset, so that arm is sent a rewritten

schema carrying what the subset can express; every constraint the rewrite removes is still checked when the response is parsed. Getting this wrong is silent, and Section 5.6 reports the campaign where we did.

Local models are served by Ollama on a single RTX 5070 Ti under WSL2. We record the model digest for every arm, because an Ollama tag is mutable, and read the serving device from the server before each timed run. The same server silently reloaded the same model onto the CPU between two campaigns, at a sixteenth of the token rate, changing every wall-clock number while leaving the actions intact. Timings below come only from runs whose device was confirmed.

4.7 Metrics

Action accuracy is the fraction of the 144 requests on which the arm returns the gold action. It is the headline number and the one that hides the most, so we report four others beside it. **Task completion** is strictly stronger — the gold action *and* the corresponding work, a certified schedule or a concrete field asked about — and equals action accuracy for every arm here, so we do not table it. It does not check the certified schedule field by field against the intent, so the two metrics are not a joint success rate.

Field exact match is a field-level rate whose denominator is fixed by the gold and does not move with what an arm answered: the seven parsed contract fields on each of the 44 gold-EXECUTE items, 308 slots for every arm, so 0.946 for the ungated frontier arm is 287 of 308. An arm answering something other than EXECUTE parsed no fields and *scores zero on all seven* rather than having them excluded, because a denominator that shrank when an arm refused would reward refusing. Three further gold fields (*topology_id*, *working_period*, *communication_range*) are bound or echoed by the runner rather than parsed, and *analysis_v36/summary.json* reports them separately. **False acceptance**, the safety metric, counts items answered EXECUTE whose gold action was not EXECUTE, plus items answered EXECUTE with an uncertified schedule, over all 144 requests rather than over the 100 that should not be executed. **Paraphrase consistency** is the fraction of the 72 groups whose two wordings received the same action.

We also report retries, model calls, token totals and mean wall time per request, together with a failure taxonomy separating semantic errors from schedule-validation failures and infrastructure failures, so that a slow or failing provider cannot be read as a model result.

4.8 Statistics

Arms are compared with a paired bootstrap resampling with replacement 10 000 times, reporting the mean difference in action accuracy with a 95% percentile interval, and no *p*-values. Two choices decide what that interval covers. Pairing is by request, but the *resampling unit is the paraphrase group*: a group is one intent written two ways, so resampling its requests separately would count one intent twice and report an interval narrower than the evidence supports. Clustering draws groups with replacement and takes both wordings of each; it widens our intervals by 1.3 to 1.4 and changes no conclusion here. And an arm’s score on a request is the *fraction of its three repeats* that answered correctly, so an item it gets right

twice contributes 2/3 rather than whichever way one run fell; the bootstrap then resamples groups over those run-averaged per-item differences. The estimand is the difference the arms would show on a fresh draw of intents *holding the runs fixed*. It is not a hierarchical interval and adds no run-to-run variance component; the spread the runs showed is reported separately, as the ranges in Table 3 and the item counts in Section 5.7. We report the request-level intervals in the supplementary so the two can be compared. With 144 items and differences of a few items, an interval communicates the precision honestly and a threshold test would not.

5 Results

Table 3 is generated from the analysis artefact rather than transcribed, and carries every paired difference; we quote the ones we use and take the findings of Section 1 in turn.

5.1 Main Results

Across its three repeats the ungated frontier arm answers 42, 43 and 43 of the 44 gold-EXECUTE requests correctly, the independent validator accepted every schedule it emitted, it matches the gold value on 94.6% of the fields those requests fix, and it acts when it should not on 2 of 144. A request takes about four seconds, replacing a manual loop of writing a configuration, invoking the solver and checking its output by hand.

It is weakest where a request is in scope but its parameters are wrong, 5 of the 16 gold-REJECT requests for the gated arm and 6 for the ungated, and Section 5.6 says why. And the CLARIFY capability is not the model’s: 8 of the 12 clarification items are settled by a deterministic stage, and until we repaired our own output contract the other 4 were failed by every model we tested.

Action accuracy spans 0.382 for the deterministic baseline to a mean 0.912 for the ungated frontier arm, and both frontier arms separate decisively from it (+0.5301, CI [+0.4120, +0.6412] ungated; +0.4630, CI [+0.3426, +0.5787] gated). No 14B arm does: their point estimates run from −0.0069 to +0.0972 and every interval contains zero, Qwen’s falling slightly below the baseline (54 of 144 against 55). The finding is not that language agents cannot do this task, but that the two open 14B models we tested cannot, and that the benchmark is not saturated by the model that can.

Among the arms without a frontier model the baseline is the *best* on gold-EXECUTE, 34 of 44, and the worst on the two categories that require declining: 0 of 16 gold-REJECT and 13 of 72 gold-UNSUPPORTED, against 10 and 24 for the gated 14B arm. It is not worst on everything — it ties the 14B arms on CLARIFY, which the topology gate decides, and leads every arm on paraphrase consistency. What it lacks is any means of declining an in-scope request with bad parameters. So an interval containing zero is not evidence of equivalence, and we draw none: the 14B arms differ from the baseline sharply on false acceptance, 30 and 54 against 74. The false-acceptance column, not the accuracy column, is what separates an agent from a rule.

5.2 Scope-Gate Ablation

The scope gate short-circuits 18 of the 144 requests before any model is called, identically for every enforcing arm: 11 correctly refused, 7 gold EXECUTE. In the ungated and advisory arms those

Table 3: Configuration and results for every arm on the 144 sealed requests. All arms see identical requests in identical order and run the same scope detector; the *scope gate* row says what it may do, off marking the two ablation arms and advisory the two where it annotates and the model decides. Denominators differ by row: action accuracy and false acceptance are out of 144, the indented action rows out of that action’s gold total, paraphrase consistency out of the 72 groups, and field exact match is a rate over 308 field slots (Section 4.7); model calls and mean seconds have none. Every model arm is the mean of three repeats, so in a count row an integer means the repeats agreed and a decimal that they did not, with the range given for the totals.

Arm model scope gate	Rule — enforcing	Phi4 phi4:14b enforcing	Phi4-ng phi4:14b off	Phi4-adv phi4:14b advisory	Qwen qwen2.5:14b enforcing	GPT gpt-5.5-2026-04-23 enforcing	GPT-ng gpt-5.5-2026-04-23 off	GPT-adv gpt-5.5-2026-04-23 advisory
Action accuracy	55	64	67	69	54	121.7 [121–122]	131.3 [130–132]	129.0 [128–130]
EXECUTE	34/44	22/44	29/44	29/44	27/44	34.7/44	42.7/44	43/44
CLARIFY	8/12	8/12	8/12	8/12	8/12	12/12	12/12	12/12
REJECT	0/16	10/16	10/16	10/16	8/16	5.0/16	6/16	4.3/16
UNSUPPORTED	13/72	24/72	20/72	22/72	11/72	70/72	70.7/72	69.7/72
Field exact match	0.351	0.549	0.685	0.685	0.646	0.792 [0.760–0.818]	0.946 [0.932–0.955]	0.952 [0.951–0.955]
False acceptance	74	30	33	30	54	3.3 [3–4]	2	3.7 [3–4]
Paraphrase consistency	70/72	45/72	46/72	46/72	61/72	68/72	69/72	70/72
Model calls	0	138	156	156	138	128	145	145
Mean seconds	0.02	1.41	1.58	1.58	1.61	3.76	4.01	3.76

Table 4: Paired differences in action accuracy, 10,000 bootstrap resamples over the 72 paraphrase groups. Intervals excluding zero are in bold.

Comparison	Difference	95% CI
GPT-adv minus Rule	+0.5139	[+0.3958 , +0.6273]
GPT minus Rule	+0.4630	[+0.3426 , +0.5787]
GPT-ng minus Rule	+0.5301	[+0.4120 , +0.6412]
Phi4-adv minus Rule	+0.0972	[−0.0208, +0.2153]
Phi4 minus Rule	+0.0625	[−0.0417, +0.1667]
Phi4-ng minus Rule	+0.0833	[−0.0417, +0.2014]
Qwen minus Rule	−0.0069	[−0.0903, +0.0764]

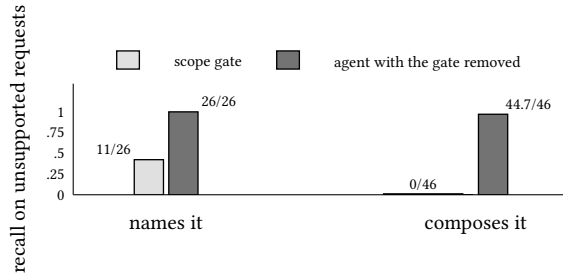


Figure 4: Recall on the 72 gold-UNSUPPORTED requests, split by a vocabulary test, independent of the gate’s own patterns, for whether the request names the out-of-scope capability. The gate’s patterns are fixed, so its bars are one number; the agent decodes at temperature 1, so its bars are the mean of the three repeats of Table 3. The gate’s rules are proximity windows written to catch a capability expressed without its name, and they fired on none of the 46 requests on the right. Per-group request ids are in figure_data/fig2_phrasing.json.

same 18 reach the model, which is what the ablation moves. These are hard negatives that name an out-of-scope concept while asking for something supported.

For the 14B agent the trade is close to even. Removing the gate recovers the 7 over-blocked items, loses 4 of the 11 correct refusals to the model’s own judgement, and moves action accuracy by an amount whose interval contains zero. What the gate buys there is safety: false acceptance is lower with it, 30 against 33.

For the frontier agent the same removal improves every accuracy figure and lowers false acceptance too: action accuracy from a mean 121.7 to 131.3 of 144 (+0.0671, CI [+0.0208, +0.1227]), gold-EXECUTE from 34.7/44 to 42.7/44, field exact match from 0.792 to 0.946, unsupported-capability recall from 70 of 72 to a mean 70.7, and false acceptance from a mean 3.3 to 2. In the repeat we inspected item by item the two false-acceptance sets overlap in only one item, so the ungated arm trades a false acceptance rather than removing one. What the gate buys is efficiency: 128 calls against 145, 3.76 seconds against 4.01.

Both frontier false-acceptance counts come from one contested class: every false acceptance the gated and the ungated arm commit is one of the four requests whose stated communication range contradicts the bound topology (Section 5.6). Dropped, both are at zero, so the two arms differ in accuracy and not in measured false acceptance; the 14B counts move from 30 against 33 to 28 against 31. Under four alternative labellings of the contested items, every paired difference in this section and the next keeps its sign and its verdict on zero, and no arm’s accuracy moves by more than 0.06 (sensitivity_labels/summary.md in the release). The direction is not uniform across arms, which is why it is worth running: the channel-conflict relabelling adds to every frontier arm and subtracts from every 14B one, because the 14B arms answer REJECT there and the frontier arms ask which count is meant. Splitting the change by whether the gate fired separates what it caused from what it did not. On the 18 requests it short-circuits, the ungated arm gains the 7 over-blocks and loses nothing, identically in all three repeats: that part is the gate’s and it is deterministic. On the other 126 the ungated arm gains three to five items and loses zero to two, never the same twice — the model sampling at temperature 1, which we report rather than fold into the gate’s effect. It recognises out-of-scope requests the gate’s patterns miss and does not need the ones it catches.

The mechanism is visible in the gate’s own recall (Figure 4). We split the 72 unsupported requests by whether they use the vocabulary of the capability they ask for: on the 26 that do, the gate catches 11 and the ungated agent all 26 in every repeat; on the 46 that do not, the gate catches *none* and the agent a mean 44.7. The gate’s recall is a property of the phrasing rather than of the request, and not for want of trying: the rules are proximity windows written to catch a capability expressed without its name. This is not the familiar observation that a keyword matcher misses paraphrases. A model weak enough to be fooled by the same phrasing is helped by it, and a model that reads the request is not, and pays for the over-blocks.

A scope gate calibrated against a weak agent buys a different trade once the agent answers the matched requests by itself: here the gate’s refusals stop being ones the agent would have missed and start being ones it would have answered correctly. Whether that is a net loss depends on what a false acceptance costs against a false refusal, which this benchmark does not price. This is one gate on one benchmark at two capability levels, so it demonstrates that the safety trade can reverse, not where it reverses.

5.3 Moving the Gate’s Authority

A short-circuiting gate’s over-blocks are unrecoverable by construction: no later stage sees the request. So we ran a third configuration in which the same detector runs and what it finds reaches the model as a non-binding notice naming the matched capability and the text it matched. Only the authority moves.

The 18 matched requests divide into 7 the gate refuses wrongly and 11 it refuses rightly, and the three configurations cover them differently. In every repeat the enforcing arm keeps all 11 correct refusals and recovers none of the 7, the ungated arm recovers all 7 and keeps 7 of the 11, and the advisory arm recovers all 7 and keeps 9 — the model overruling the notice where the gate was wrong and following it on two requests it would otherwise have missed. For the 14B agent that is +0.0347 in action accuracy over the enforcing arm, CI $[-0.0208, +0.0903]$, and +0.0139 over the ungated arm, CI $[+0.0000, +0.0347]$, at the enforcing arm’s false-acceptance count of 30 against the ungated arm’s 33, and at 156 model calls against 138. Neither interval excludes zero decisively, so the observed five-item gain describes these runs and the coverage pattern is the firmer part. The two refusals it loses are ones the model overrules; the two the ungated arm additionally loses are ones it never saw flagged. Behind the frontier agent the notice changes nothing we can observe. On the 18 matched requests — the only ones whose prompt it alters — the advisory and ungated arms returned identical actions in all three repeats, both recovering the 7 over-blocks and keeping all 11 correct refusals. Against the enforcing arm the advisory arm gains +0.0509, CI $[+0.0046, +0.1065]$; against the ungated arm it differs by -0.0162 , CI $[-0.0463, +0.0046]$, and that difference falls entirely on requests the gate never matched.

What separates the two cases is not the gate’s recall, the same 11 of 72 either way, but whether the gate is right where the model is wrong: for the 14B agent the two disagree on the matched items in complementary directions, so the notice adds coverage; for the frontier agent the ungated arm already answers all 18 correctly. *An advisory gate is worth the coverage it adds where the model is already*

wrong, and for the frontier agent there was none left to add. That is a statement about two agents, not a trend: a stronger model can change which items it gets wrong as well as how many.

5.4 Retry Ablation

Setting the consistency check’s retry budget to zero, changing nothing else, costs the frontier agent seven items of action accuracy (-0.0486 , CI $[-0.1042, +0.0000]$) and drops field exact match from 0.799 to 0.666. The accuracy interval reaches zero under group resampling, so only the field-level effect is separable from sampling.

All seven are gold-EXECUTE items, and 92 of the 94 retries across all arms were triggered by the same condition, the model changing a field the user had fixed. Retry buys field fidelity; on action choice our evidence does not separate it from zero. It also scales with the model: of the requests that triggered a retry, the 14B agent recovered 11%, the second local model 38%, and the frontier agent 73%. A retry budget is worth more to an agent that can use the diagnosis.

5.5 Direct Generation Baseline

The architecture assumes a deterministic solver is necessary, so we asked the model for the schedule itself on 23 gold-EXECUTE requests balanced over the seven topology sizes, selection fixed before the run, giving it the full topology, the constraints in the validator’s terms and one correction round. Eleven were valid, three at once and eight after correction, and every one is worse than the solver’s on the same request — $1.4\times$ its span at thirty nodes, $789\times$ at three hundred, where the validator certified 22,887 slots against the solver’s 29. *Constraint satisfaction and objective quality come apart here, and the validator sees only the first.* This is what one prompting of one model on 23 items did, not a bound on what a language model can schedule; the per-size ratios, the 23 items and the prompt are in `analysis_b2/direct_generation.json`, the ledger it names and the released scripts/. A validator cannot object, because a validator checks feasibility and not quality.

Nothing in the model’s output distinguishes its correct schedules from its incorrect ones, or its efficient ones from one 789 times too long: the checker bounds correctness, and only the solver bounds quality.

The architecture assumes a deterministic solver is necessary. That assumption is testable: ask the model for the schedule itself and see what comes back. We run this on the 23 gold-EXECUTE requests that a balanced sample over the seven topology sizes selects, with the selection fixed before the run and recorded in the released artefact.

The baseline is given every advantage we can give it: the full topology the solver sees, the constraints stated in the terms the validator checks, the same schema-constrained adapter the agent uses, and one correction round in which the validator’s violations are handed back to it — more help than the agent’s own retry receives. The agent is never told what a validator found.

Three of the 23 schedules were valid on the first attempt. Eight more became valid after the validator told the model what was wrong, for 11 of 23 in total. The failures are not wild: 22 of the 23 responses contained exactly the right number of transmissions. The median failing attempt violated two constraints out of two

Table 5: Direct generation, by network size. *Valid* counts schedules the independent validator certified; *first* counts those certified without the correction round. *Latency* is the ratio of the direct schedule’s span to the solver’s on the same request, over the certified schedules.

Nodes	Items	Valid	First	Latency vs solver
30	3	1	1	1.4×
50	4	4	0	1.4–6.6×
100	4	2	0	13.3–14.4×
150	3	2	2	22.0–22.4×
200	3	1	0	54.0×
250	3	0	0	—
300	3	1	0	789.2×
all	23	11	3	median 13.3×

hundred-odd, most often by scheduling a transmission between nodes that are not neighbours or by having a parent transmit before its children.

The latency column is the part that settles the question. Every certified direct schedule is worse than the solver’s on the same request, from 1.4× at thirty nodes to 789× at three hundred, and the penalty grows with network size. The worst case is worth stating plainly, because it is not a near miss: on a 300-node network the model produced a schedule the validator certified, using 22,887 slots where the solver needs 29. It satisfied every constraint by giving almost every transmission a slot of its own. *It met the constraints by abandoning the objective*, and a validator cannot object, because a validator checks feasibility and not quality.

So the three components answer three different questions, and none of them substitutes for another. Without a validator, an agent that generates schedules directly is right 3 times in 23, while 22 of its 23 answers *look* right — correctly formatted, correctly sized, and wrong. With a validator in the loop, it reaches 11 of 23, which is a coin flip. And a valid schedule is not a good one: the solver is what turns “occasionally feasible after correction” into “always feasible, and short”. The deepest of the three is the first. Nothing in the model’s output distinguishes its correct schedules from its incorrect ones, or its efficient ones from a schedule 789 times too long. That is the argument for putting a deterministic solver and an independent checker around a language model, and it is an argument about the *absence of a reliability signal* rather than about the model being weak.

Direct generation is also expensive where the architecture is cheap. Producing a schedule for a 300-node network takes the model on the order of fifteen thousand completion tokens, against roughly a thousand for the structured request the agent emits, and the baseline averaged 281 seconds per request against the agent’s four. The whole baseline cost 519,601 completion tokens over 43 attempts for 23 requests.

5.6 Error Attribution

The gated frontier arm gets 22 to 23 items wrong across its three repeats, and in each run 11 are forced by the architecture rather than chosen by the model: 7 gate over-blocks and 4 requests the topology gate terminates, the same items each time because the stages producing them run before the model.

The sharpest case is CLARIFY, where an earlier version of our own pipeline made the capability unmeasurable. Of the 12 CLARIFY items the 8 the topology gate decides were answered correctly by every arm, the baseline included; the 4 requiring the model to formulate the question were failed by every arm — not because the models missed the ambiguity but because they identified it and our output contract discarded the answer. The transcripts show the correct question being asked and rejected for its shape.

The cause was a silent degradation in the provider layer: the API’s strict mode rejects the agent’s decision schema outright, our client caught the rejection and retried without one, and an entire arm ran unconstrained. The schema carries bounds, defaults and bare const values, which the local server enforces directly as a generation grammar. Unconstrained, the model returned the right decision in the wrong shape: field names it invented, and questions as bare strings where objects were required. Each was rejected by the parser, and after the retry budget the decision was recorded as a refusal.

Sending the API a schema rewritten into the subset it accepts repairs this, and the repair is confined to the agent: the sealed requests, gold labels, prompts and topologies keep their hashes, and all 4 items then pass in both frontier arms. *An output contract strict enough to reject a correct answer is recorded by a benchmark as a model failure*. Nothing in the results of the broken run looked wrong.

The repair also lowered gold-REJECT for the frontier arms, from 11 of 16 to a mean 5.0 for the gated arm and from 10 to 6 for the ungated one. Eight requests account for it, four of each of two kinds, named here so the claim can be checked against the released ledgers.

Four state one channel count in prose and a different one in an adjacent run-settings block: ft3-r0004 and ft3-r0113, one paraphrase group, ft3-r0112 and ft3-r0121, another. The repaired agent asks which count is intended where the gold says refuse, in every repeat but one, where the gated arm refuses ft3-r0113. We read that as a defensible disagreement about the label rather than an error.

The other four are a blind spot the repair uncovered: ft3-r0033, ft3-r0106, ft3-r0130 and ft3-r0141 each state a communication range of 18.0 against a bound topology whose catalogue value is 20.0, and the repaired agent takes the topology’s value. ft3-r0033 executes in every repeat of both arms; the other three split between them. The unrepaired agent refused them only because it placed `communication_range` in its request, which is not a field of the request schema, and the unknown-field error surfaced as a refusal. Those refusals were accidents. With the contract enforced the gap is visible: no stage of the pipeline compares a stated communication range against the bound topology, so this class of contradiction cannot be caught at all. We report it as an open defect rather than repair it after seeing the results.

5.7 Reproducibility

Within one serving session the local arms are reproducible item by item: across three repeats of each of the four, every arm returned the same action on all 144 requests, so every metric computed from the actions has a range of zero, as do model calls and tokens. Wall

time is not invariant. Across sessions the same arm drifts by two to four items, which is why every local arm in Table 3 comes from one session. An earlier campaign whose repeats were spread over several sessions showed a 14B arm at 64, 62 and 62 of 144 under settings it reported as identical. A difference smaller than that drift is not interpretable from a single run. The frontier arms are repeated three times as well, but they decode at temperature 1 because the provider accepts nothing else, so their repeats sample the model rather than the serving stack; they span one to two items of 144.

This is where the second silent failure showed. Between two campaigns the local server reloaded the same model onto the CPU at a sixteenth of the token rate, changing every wall-clock number by an order of magnitude while leaving all 144 actions intact — wrong in exactly the column nothing else flags. Hence the device check before each timed run.

6 Discussion and Conclusion

The gate we measured is the kind that gets written first, and its ordering — matched against the raw request before the model runs — is sound. What the results do not support is the assumption that such a gate is monotone. Section 5.2 shows it costs both agents action accuracy and that what reverses is safety: false acceptance falls for the 14B agent, 30 against 33, and rises for the frontier agent, a mean 3.3 against 2. Its over-blocks are paid for by refusals the 14B agent would have missed, and are not paid for behind the frontier agent, which answers the matched requests itself. Whether the first trade is worth making depends on the price of a false acceptance against a false refusal, which this benchmark does not set; the ablation settles only that the two agents are not on the same side of it. Two consequences follow. A scope gate should be re-measured by ablation at every model upgrade, which can turn a safety component into an accuracy cost charged to the model. And what an upgrade should change first is the gate’s *authority* rather than its detection: a short-circuiting gate’s over-blocks cannot be recovered by any later stage, while demoting the same detector to an advisory notice recovers them for the 14B agent at no cost in false acceptance (Section 5.3) and changes no observed decision behind the frontier one. Demotion is an alternative worth evaluating alongside removal. We tested one detector on one task family and priced no errors, so a deployment with asymmetric costs may order the three modes differently.

The architecture’s guarantee is narrow and it held: no schedule reached the output in any arm, in any run, without a certificate from a validator that shares no code with the scheduler, and none was rejected. It is conditional on that validator being correct on the conditions of Section 3.1, which we test by second implementation and mutation rather than prove, and which held across 1260 topology–channel configurations of the core alone. What it does not cover is where our errors live: the wrapper certifies the schedule against the request the agent constructed, not that the construction is faithful to the user. The blind spot of Section 5.6 is that in miniature — a stated communication range contradicting the bound topology passes every check we have, because the contradiction is between the user’s words and the orchestrator’s binding, which no component compares. The checks an architecture is most likely to

be missing are those spanning a boundary, because each component is individually correct.

Three of our findings were available only because the harness recorded more than the agent’s answer; all three would otherwise have been charged to the model. The 18 items the scope gate refuses are indistinguishable, in the answer alone, from items the model refused; recording the gate’s verdict per request is what makes the ablation interpretable at all. The CLARIFY collapse looked exactly like four models failing the same four items, and was visible as our own contract’s fault only because every rejected attempt was logged with its error code and raw payload. The structured-output fallback produced a complete, plausible, entirely unconstrained arm that nothing in the answers marked; it was recoverable only because the client stamped each call with the mode it had actually negotiated — a field added for provenance, not for debugging. So we state a requirement for this kind of evaluation: a ledger should record, per request, which deterministic stage terminated it, the decoding mode negotiated with the provider and the text of every rejected attempt, and, per run, the serving device — all cheap at write time, none recoverable afterwards. A benchmark that stores only the final action can report a pipeline defect as a model capability limit, and be internally consistent while doing it.

We set out to replace a configuration file with a sentence, and the agent does that; saying so required an evaluation separating its failures from its scaffolding’s, which is where the findings came from. The benchmark separates sharply, 0.382 for a deterministic baseline to 0.912 for the best agent, and neither 14B model separates from that baseline. The evidence has limits: one task family, one regular-expression detector, no arm between 14B and frontier, unmatched decoding (Section 4.6), half the benchmark written in a session that had also built the pipeline (Section 4.4), a repair made after seeing failures on the sealed set so that nothing here is held out, four contested label groups whose effect on every comparison we measure (Section 5.2) and one unfixed defect. This is a controlled case study, not a general result about scope gates. What transfers is about evaluation: a benchmark score is a property of the whole pipeline, which fails in ways that leave no trace in the answers.

References

- [1] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. 2018. Safe Reinforcement Learning via Shielding. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*. AAAI Press, New Orleans, Louisiana, USA, 2669–2678. arXiv:1708.08611.
- [2] Asger Horn Brorholt, Kim Guldstrand Larsen, and Christian Schilling. 2025. Compositional Shielding and Reinforcement Learning for Multi-Agent Systems. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. IFAAMAS, Detroit, Michigan, USA, 399–407. arXiv:2410.10460.
- [3] Liang Cheng, Mingsheng Cai, Jiuming Jiang, and Luo Mai. 2026. Do Agents Know What They Can’t Do? Evaluating Feasibility Awareness in Tool-Using Agents. arXiv:2605.28532
- [4] Francesca Da Ros, Luca Di Gaspero, Michael Soprano, and Kevin Roitero. 2025. Large Language Models for Combinatorial Optimization: A Systematic Review. To appear in *ACM Computing Surveys*, doi:10.1145/3801961. arXiv:2507.03637
- [5] Saibo Geng, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. JSON-SchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. arXiv:2501.10868
- [6] Xia Jiang, Jing Chen, Cong Zhang, Jie Gao, Chengpeng Hu, Chenhao Zhang, Yaixin Wu, and Yingqian Zhang. 2026. Reasoning in a Combinatorial and Constrained World: Benchmarking LLMs on Natural-Language Combinatorial Optimization. arXiv:2602.02188

- [7] Fangzheng Li, Aimin Zhang, and Chen Lv. 2026. Constraint Tax in Open-Weight LLMs: An Empirical Study of Tool Calling Suppression Under Structured Output Constraints. *arXiv:2606.25605*
- [8] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. *arXiv:2507.21504*
- [9] Gerasimos Papanikolaou-Ntais, Alexandros Kalokylos, and Athanasios Kanavos. 2026. Agentic-V2X: Small Language Model Agents for Deadline-Aware V2X Scheduling in 5G/6G Networks. *arXiv:2607.04290*
- [10] Traian Rebedea, Razvan Dinu, Makes Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Singapore, 431–445.
- [11] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Mexico City, Mexico, 5377–5400.
- [12] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julien Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. 2023. NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for each Benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 10776–10787. *arXiv:2310.18018*.
- [13] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. *arXiv:2408.02442*
- [14] Jingwen Tong, Wei Guo, Jiawei Shao, Qiong Wu, Zijian Li, Zehong Lin, and Jun Zhang. 2025. WirelessAgent: Large Language Model Agents for Intelligent Wireless Networks. *arXiv:2505.01074*
- [15] Van-Vi Vo, Duc-Tai Le, Syed M. Raza, Moonseong Kim, and Hyunseung Choo. 2024. Active Neighbor Exploitation for Fast Data Aggregation in IoT Sensor Networks. *IEEE Internet of Things Journal* 11, 8 (2024), 13199–13216.